



INSTITUTO FEDERAL
Sul-rio-grandense

Câmpus
Camaquã

EDUCAÇÃO
PÚBLICA
100%
GRATUITA

PROGRAMAÇÃO ORIENTADA A OBJETOS

Introdução



Prof. Luiz Wasielewski



POO

Introdução





P OO

O que é?

A Programação Orientada a Objetos (P OO) é um **paradigma de programação** que utiliza a abstração para criar modelos baseados no mundo real. Diferente do paradigma procedural, focado em funções e sequências de passos, a P OO organiza o software em torno de **dados** (objetos) em vez de lógica puramente funcional.



POO

Pilares





POO

Pilares

- ❖ **Abstração:** Capacidade de isolar os elementos relevantes de um objeto ou problema do mundo real para o domínio do software
- ❖ **Encapsulamento:** Técnica de “ocultar” os detalhes internos de um objeto e expor apenas o necessário - promove segurança contra acessos indevidos e erros acidentais.



POO

Pilares

- ❖ **Herança:** Permite que uma classe (subclasse) herde atributos e comportamentos de outra classe (superclasse).
- ❖ **Polimorfismo:** Muitas formas. É a capacidade de um objeto ser tratado como um tipo mais genérico, mas executar comportamentos específicos de sua subclasse.



Classe

Definição

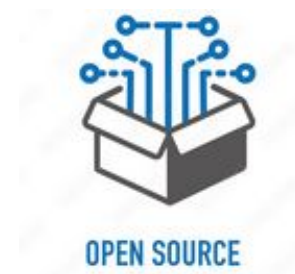
Classe não é objeto, mas um “molde” para criá-lo. Ela define quais atributos (características) e métodos (comportamentos) os objetos desse tipo possuirão, mas não representa um objeto em si.



Codificação

Linguagem de programação

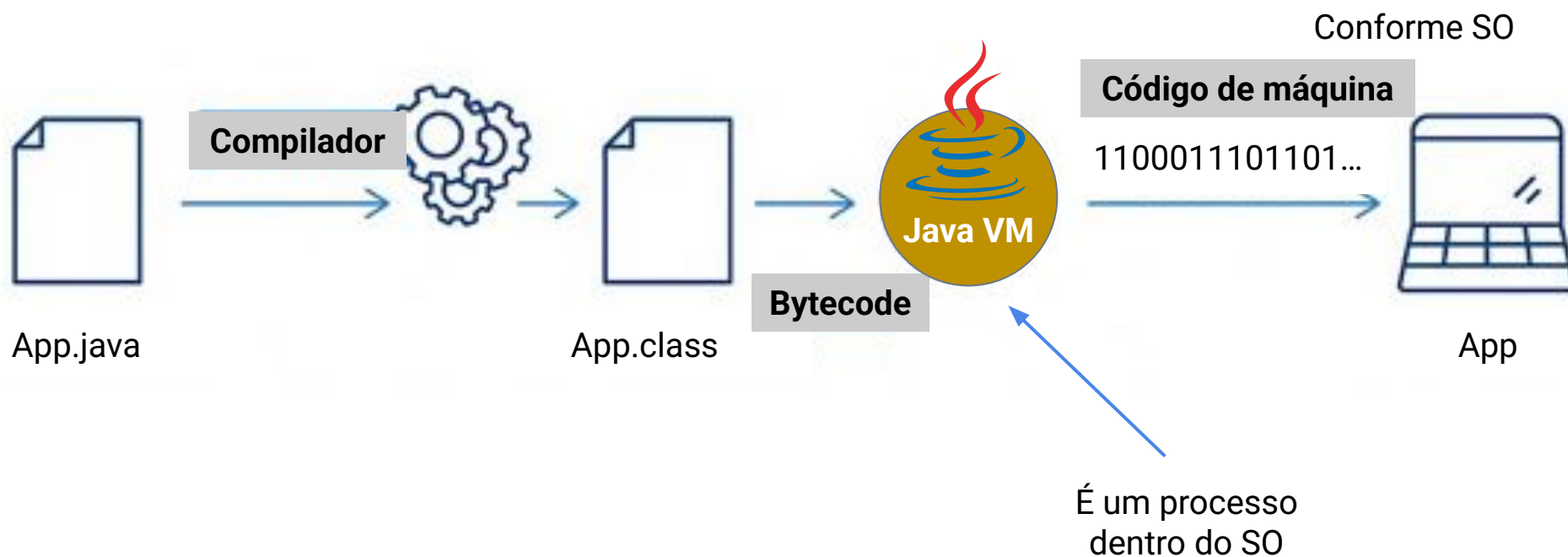
Nas aulas utilizaremos como linguagem de programação o Java. Nosso ambiente de desenvolvimento é o netbeans.





Java

Máquina Virtual Java - JVM





Classe

Codificação

Visibilidade

Nome da classe

`public class Veiculo {`

`String modelo;
String cor;
int velocidadeMaxima;`

Atributos ou propriedades
da classe

`}`



Objeto

Definição

Um objeto é uma **instância** física ou concreta de uma classe. Enquanto a classe é abstrata, o objeto ocupa memória e possui valores reais atribuídos aos seus campos.

- **Estado:** definido pelos valores de seus atributos em um determinado momento;
- **Comportamento:** definido pelos métodos que o objeto pode executar;
- **Identidade:** Cada objeto é único, mesmo que possua os mesmos dados de outro.



Objeto

Como instanciar

```
public class Principal {  
  
    public static void main(String[] args) {  
        Veiculo carro = new Veiculo();  
  
        carro.cor = "verde";  
  
    }  
}
```

Tipo de Dado

Construtor utilizado

Acessando propriedades
do objeto instanciado



Classe

Construtor

O construtor é um método padrão especial utilizado para inicializar os objetos de uma classe. Ele é executado automaticamente no momento em que a palavra-chave **new** é utilizada.

- ❖ **Propósito:** Garantir que o objeto comece sua existência em um estado válido, possibilitando atribuir valores iniciais aos atributos.
- ❖ **Regras:** Segue o mesmo nome da classe, e não possui retorno de dado.
- ❖ **Construtor padrão:** Se nenhum construtor for declarado, o java cria um vazio automaticamente.



Classe

Construtor

```
public class Veiculo {  
  
    String modelo;  
    String cor;  
    int velocidadeMaxima;
```

Construtor

```
    public Veiculo(String modelo) {  
        this.modelo = modelo;  
    }
```

Referência ao
atributo da classe

Parâmetro



Classe

Sobrecarga de Construtores

```
public class Veiculo {  
    // atributos da classe
```

```
    public Veiculo(String modelo) {  
        this.modelo = modelo;  
    }
```

Construtor 01

```
    public Veiculo(String modelo, String cor) {  
        this.modelo = modelo;  
        this.cor = cor;  
        this.velocidadeMaxima = 220;  
    }
```

Construtor 02

```
}
```



Classe

Destrutores

"Diferente de linguagens como C e C++, o Java não possui destrutores para liberar memória manualmente. Essa tarefa é do **Garbage Collector** (Coletor de Lixo), que roda em segundo plano na JVM e remove os objetos inativos automaticamente.

Ainda assim, é possível “sinalizar” quando um objeto não for mais necessário, atribuindo um valor “**null**” à variável de referência:

```
Veiculo carro = new Veiculo();  
carro = null;
```




Classe

Finalizadores

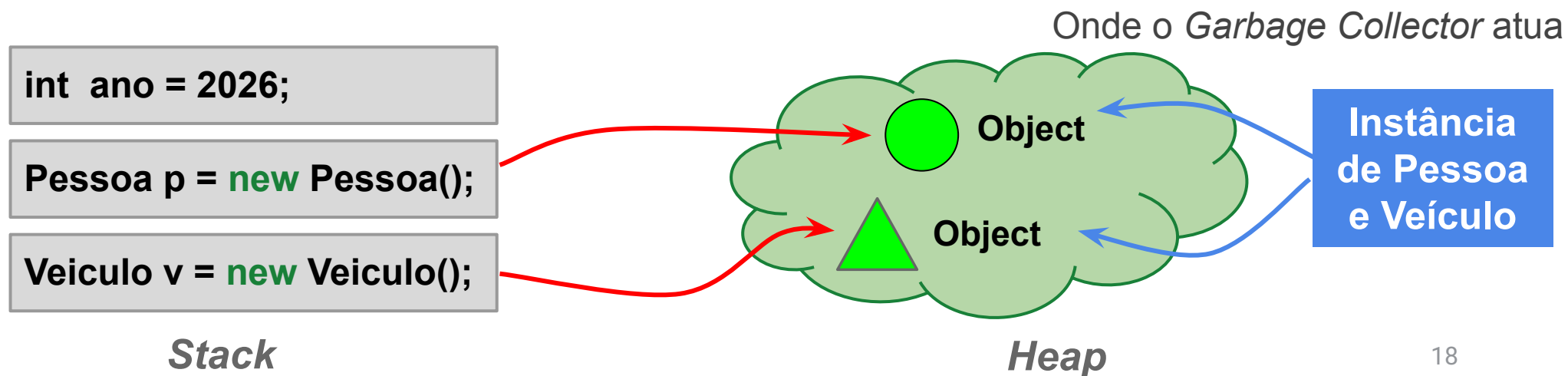
Para fins de conhecimento, cabe ressaltar que versões mais antigas do Java disponibilizavam um método finalizador ***finalize()*** para liberar recursos do sistema (como fechar arquivos abertos ou encerrar conexões com banco de dados) antes que o objeto fosse destruído pelo Garbage Collector. Nas versões atuais da linguagem essa prática é descontinuada.



Alocação de Memória

Heap e Stack

- ❖ **Stack (Pilha):** memória de curto prazo. Armazena as chamadas de métodos, variáveis de tipos primitivos e variáveis de referência dos objetos. Possui escopo local e acesso de alta velocidade.
- ❖ **Heap (Monte):** área de alocação dinâmica. Local onde os objetos e seus atributos ganham vida e ocupam espaço.





Classe

Métodos (comportamento)

Definem o comportamento dos objetos de uma classe, isto é, as ações que o objeto pode realizar ou as mensagens que ele pode processar.

- ❖ **Funcionalidade:** pode manipular os atributos internos do objeto para alterar seu estado ou realizar cálculos.
- ❖ **Assinatura:** composta por um nome, tipo de retorno, e opcionalmente, parâmetros de entrada.



Classe

Métodos (comportamento)

```
public boolean acelerar(int incremento) {  
    //validações  
    this.velocidadeAtual += incremento;  
    return true;  
}
```



Objeto

Como utilizar

```
public class Principal {  
  
    public static void main(String[] args) {  
        Veiculo carro = new Veiculo("modelo");  
  
        if (carro.acelerar(10)) {  
            System.out.println("O carro está acelerando...");  
        };  
  
    }  
}
```

Altera comportamento



Exercícios Moodle



Chegou a sua vez, mão na massa!



POO

Encapsulamento



Encapsulamento



Encapsulamento

Definição

O encapsulamento é a técnica de “ocultar” os detalhes internos de funcionamento de um objeto, expondo para o mundo exterior apenas o que é estritamente necessário para sua utilização.



Encapsulamento

Por que encapsular?

Podemos destacar alguns benefícios:

- ❖ **Proteção e segurança:** protege os dados internos do objeto contra acessos indevidos ou modificações acidentais.
- ❖ **Interface de comunicação:** uma interface pública de interação (métodos) é definida para acessar e manipular os atributos (propriedades) protegidas do objeto.
- ❖ **Independência de Implementação:** permite modificar a lógica interna da classe sem impactar quem a utiliza, desde que a interface pública (métodos de acesso) permaneça igual.



Encapsulamento

Modificadores de acesso

- ❖ **default** (padrão): Quando não definimos nenhum modificador de acesso, por padrão, o java atribui o ***package-private***, ou seja, o acesso aquela propriedade é permitido apenas a classes dentro do mesmo pacote.

```
public class Veiculo {  
  
    String modelo;  
    String cor;  
  
}
```

**Acesso default
(mesmo pacote)**



Encapsulamento

Modificadores de acesso

- ❖ **public** (público): o membro é acessível de qualquer lugar, por qualquer outra classe, independentemente do pacote. É usado para definir a **interface de comunicação** do objeto.

```
public class Veiculo {  
  
    public String modelo;  
    public String cor;  
  
}
```

Atributos públicos,
visibilidade irrestrita



Encapsulamento

Modificadores de acesso

- ❖ **protected** (protegido): o membro é acessível para classes no mesmo pacote e para subclasses (através de herança), mesmo que em pacotes diferentes.

```
public class Veiculo {  
  
    protected String modelo;  
    protected String cor;  
  
}
```

Atributos restritos



Encapsulamento

Modificadores de acesso

- ❖ **private** (privado): o membro é acessível apenas dentro da própria classe onde foi declarado. É o nível mais restritivo e a base para proteger os dados contra modificações acidentais.

```
public class Veiculo {  
  
    private String modelo;  
    private String cor;  
  
}
```

Atributos **TOTALMENTE**
restritos. Requer interface
pública



Encapsulamento

Modificadores de acesso - *private*

Para acessar um atributo privado de um objeto, é necessário que a classe defina uma interface de comunicação - métodos de acesso **get/set**.

```
private int velocidadeAtual;
```

O atributo a ser protegido

Método para
“pegar” o dado

```
{  
    public int getVelocidadeAtual() {  
        return velocidadeAtual;  
    }  
}
```

Método para
atribuir um valor

```
{  
    public void setVelocidadeAtual(int velocidadeAtual) {  
        this.velocidadeAtual = velocidadeAtual;  
    }  
}
```



Encapsulamento

Como utilizar

```
public class Principal {  
  
    public static void main(String[] args) {  
        Veiculo carro = new Veiculo();  
  
        carro.setVelocidadeAtual(10);  
        System.out.println(carro.getVelocidadeAtual());  
    }  
}
```

Atribui valor ao
atributo do objeto

Obtém o valor do
atributo do objeto



Exercícios Moodle

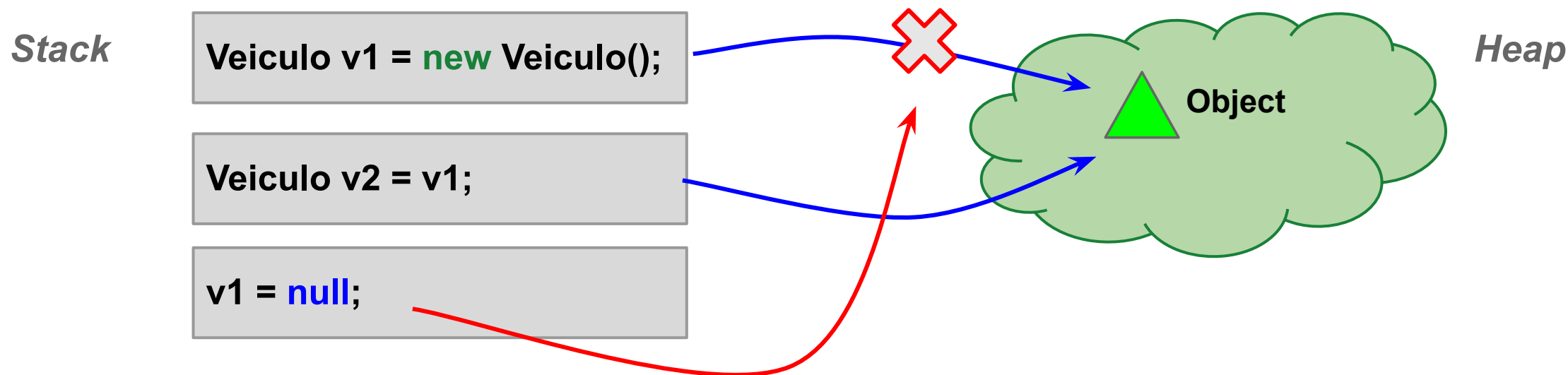


Chegou a sua vez, mão na massa!



Mais sobre Objetos

Variável referência, não objeto!



 Ambos os objetos apontam para o mesmo endereço de memória. O objeto “v2” não recebe uma “cópia”, mas o endereço onde “v1” está armazenado. Mesmo removendo a referência de um dos objetos, a instância continua existindo, pois “v2” guardou o caminho.



Mais sobre Objetos

Variável referência, não objeto!

```
Produto p1 = new Produto("");  
Produto p2 = p1;  
p1 = null;
```

```
System.out.println(p2.imprimirDados());
```



```
System.out.println(p1.imprimirDados());
```

Lançará uma
exceção de nulidade

```
Exception in thread "main" java.lang.NullPointerException: Cannot invoke "aux.Produto.imprimirDados"  
    at aux.Principal.main(Principal.java:25)  
Nome: p3 | Preço: 0.0 | Quantidade em Estoque: 0  
/home/dev/snap/netbeans/common/142/executor-snippets/run.xml:111: The following error occurred whil  
/home/dev/snap/netbeans/common/142/executor-snippets/run.xml:68: Java returned: 1  
BUILD FAILED (total time: 0 seconds)
```



Ainda Sobre Alocação de Memória

String Pool

String Pool (Piscina de Strings), é uma área especial dentro da memória **Heap** que armazena **apenas uma cópia de cada literal de String**. Ou seja, quando criamos uma String literal (ex.: String escola="ifsul"), o java verifica se ela já existe no *Pool*:

- Se já existe, a nova variável recebe a **referência** (endereço) do objeto que armazena "ifsul".
- Caso contrário, um novo objeto é criado no *Pool*.

Obs.: Em java String é objeto!



String Pool

Comparando Tipo e Conteúdo

Compara a referência

```
String nome = "Fulano";  
String aux = "Fulano";
```

```
if (nome==aux) {  
    System.out.println("nome==aux") ;  
}
```

```
if (nome.equals(aux)) {  
    System.out.println("nome-equals-aux") ;  
}
```

Ambas instruções
serão executadas



String Pool

Comparando Tipo e Conteúdo

Compara o conteúdo

```
String nome = "Fulano";  
String aux = new String("Fulano");
```

```
if (nome==aux) {  
    System.out.println("nome==aux");  
}
```

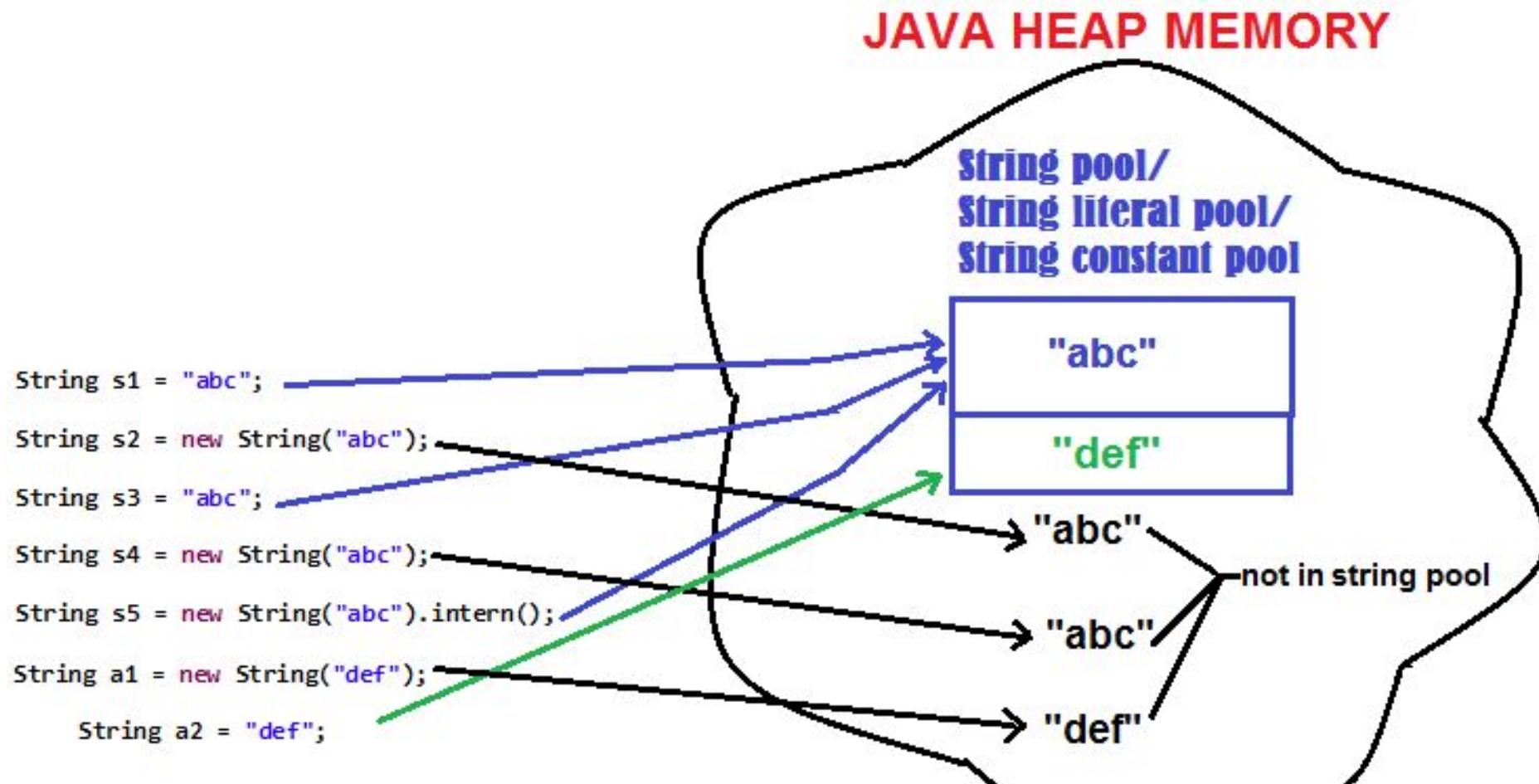
Referências
distintas

```
if (nome.equals(aux)) {  
    System.out.println("nome-equals-aux");  
}
```

Apenas esta
instrução será
executada



String Pool





Exercícios Moodle



Chegou a sua vez, mão na massa!



POO

Herança



Herança



POO

Herança

Herança é um mecanismo eficaz de **reutilização de código** em que uma classe pode **estender** outra, herdando seus atributos e métodos, e adicionando ou modificando comportamentos.

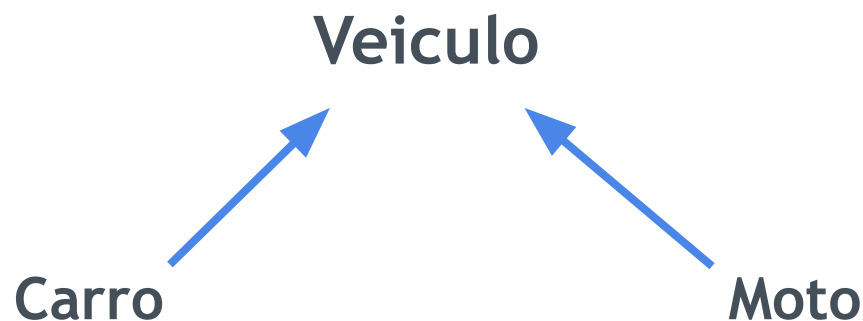
Representa uma relação de **especialização** (“é um”), cuja classe mais específica aproveita e expande uma classe mais geral. Dessa forma, criamos uma hierarquia de classes.



POO

Herança - Hierarquia

- **Superclasse** (classe pai): a classe mais genérica na hierarquia
- **Subclasse** (classe filha): a classe especializada que estende a superclasse, herdando todos os atributos e métodos.



Carro é um veículo

Moto é um veículo



POO

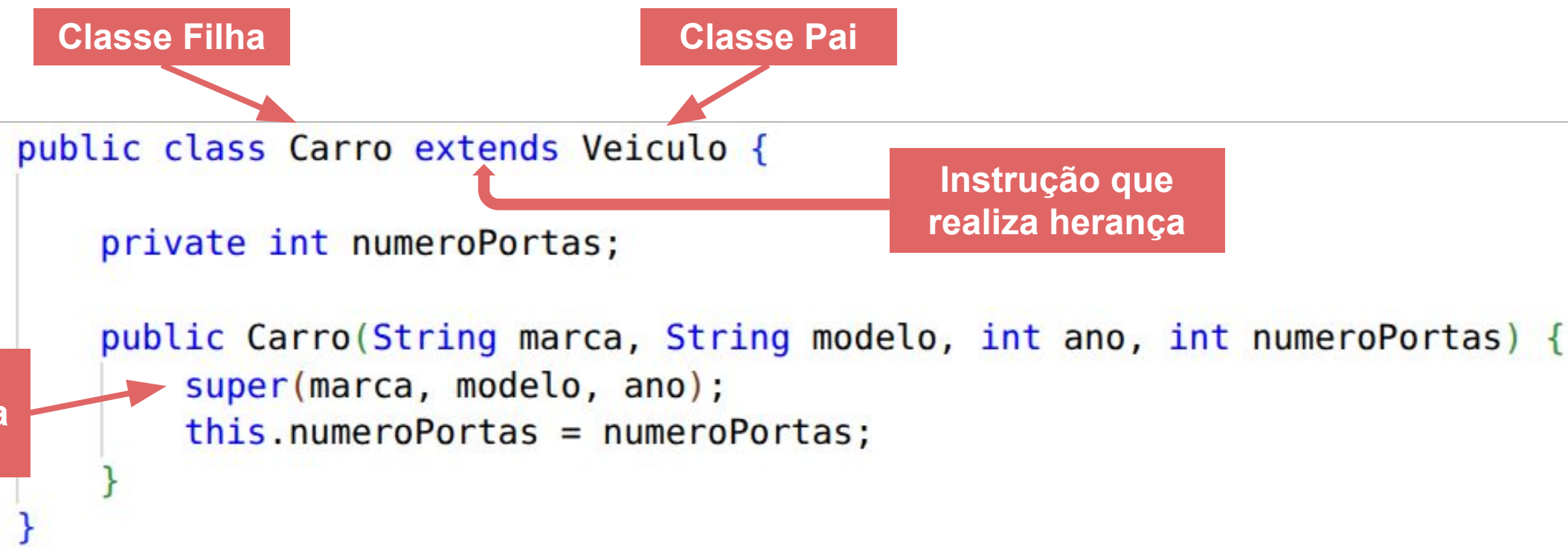
Herança - Superclasse

```
public class Veiculo {  
    private String marca;  
    private String modelo;  
    private int ano;  
  
    public Veiculo(String marca, String modelo, int ano) {  
        this.marca = marca;  
        this.modelo = modelo;  
        this.ano = ano;  
    }  
    // métodos getters e setters...  
  
    public void ligar() {  
        System.out.println("O veículo esta pronto para rodar");  
    }  
}
```



POO

Herança - Subclasse



Dizemos que, Carro **é um** veículo



POO

Herança

Mas e os comportamentos (métodos) da classe filha, serão os mesmo da classe pai?

Não, podemos sobrescrevê-los, para que assumam comportamentos distintos, específicos ao contexto.

Para isso, usamos a anotação **@override** sobre o método.

Obs.: a assinatura do método não deve ser modificada, somente a lógica interna!



POO

Herança - *@Override*

```
public class Carro extends Veiculo {  
  
    private int numeroPortas;  
  
    public Carro(String marca, String modelo, int ano, int numeroPortas) {  
        super(marca, modelo, ano);  
        this.numeroPortas = numeroPortas;  
    }  
  
    @Override  
    public void ligar() {  
        System.out.println("O " + getModelo() + " esta ligado!");  
    }  
}
```

Comportamento
diferenciado

Anotação de sobrescrita



POO

Herança - No método main()

```
public class Main {  
    Run | Debug  
    public static void main(String[] args) {  
        Carro c = new Carro(marca: "fiat", modelo: "mobi", ano: 2020, numeroPortas: 4);  
        c.ligar();  
    }  
}
```

Método executado:

```
@Override  
public void ligar() {  
    System.out.println("O carro " + getModelo() + " "+getAno()+" esta ligado!");  
}
```



POO

Herança em Java

Atenção:

Java não suporta múltipla herança, somente herança vertical.



POO

Hierarquia de Classes Java

Classe Object

- É a classe raiz do java. Dela descendem as demais.
- É este o motivo de utilizarmos *@override* em alguns métodos comuns, como o *toString()*.

Saiba mais



Exercícios Moodle

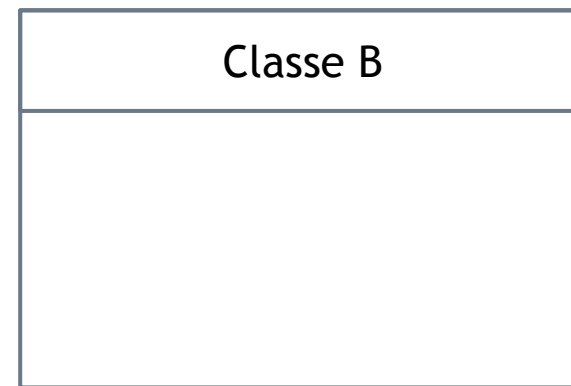
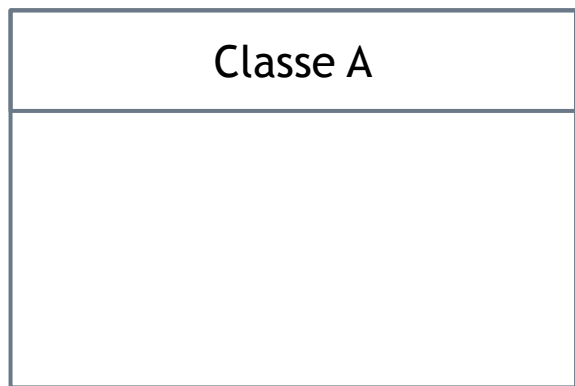


Chegou a sua vez, mão na massa!



POO

Relacionamento entre Classes





Relacionamento entre Classes

Associação

É o relacionamento mais básico. A associação ocorre quando uma classe **conhece** outra e vice-versa. Os objetos podem existir de forma independente.

Pode ser subdividido em:

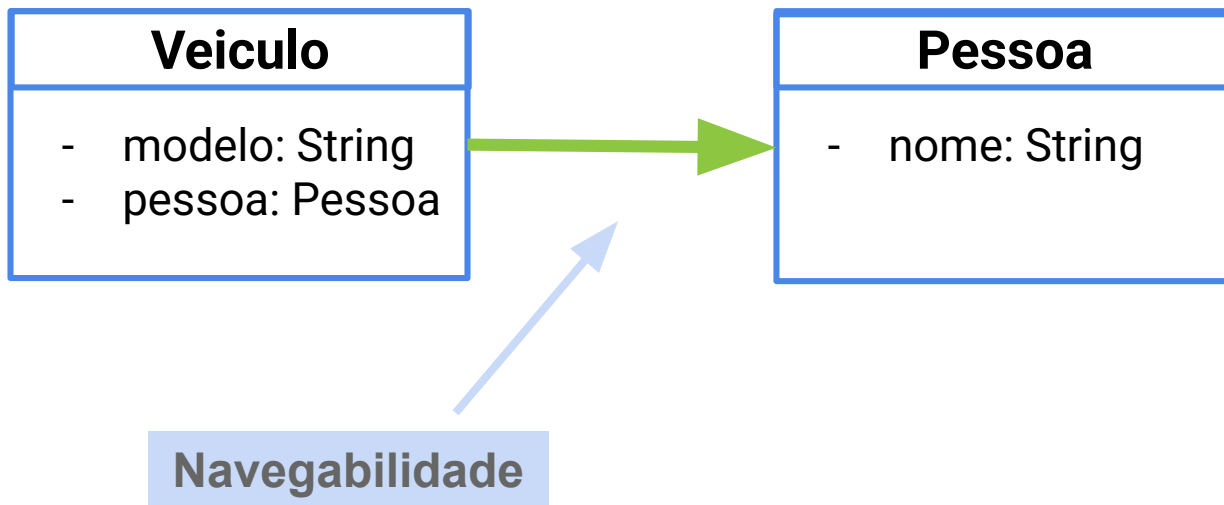
- **Unidirecional:** uma classe conhece a outra, mas não o contrário.
- **Bidirecional:** ambas as classes se conhecem.



Relacionamento entre Classes

Associação Unidirecional

- Veiculo conhece Pessoa, mas Pessoa não sabe que tem Veiculo.
- Veiculo possui uma referência para Pessoa



```
public class Veiculo {  
  
    private String modelo;  
    private Pessoa pessoa;  
  
}
```

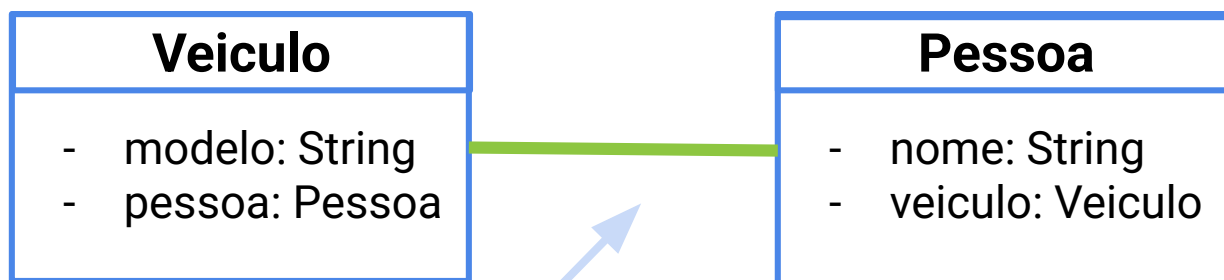
```
public class Pessoa {  
  
    private String nome;  
  
}
```



Relacionamento entre Classes

Associação Bidirecional

- Ambas as classes se conhecem.
- Veiculo conhece Pessoa, e Pessoa sabe que tem um Veiculo.
- Há uma referência de Veiculo para Pessoa, e vice-versa.



Navegabilidade

```
public class Veiculo {  
  
    private String modelo;  
    private Pessoa pessoa;  
}
```

```
public class Pessoa {  
  
    private String nome;  
    private Veiculo veiculo;  
}
```



Relacionamento entre Classes

Associação

Pergunta:

O que acontece se eu fizer `veiculo.setPessoa(null)`?



Relacionamento entre Classes

Associação

Pergunta:

O que acontece se eu fizer `veiculo.setPessoa(null)`?

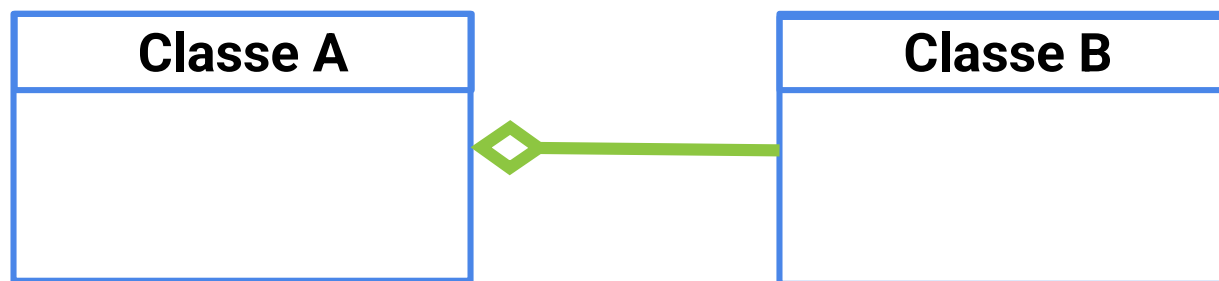
- O **atributo pessoa** do **objeto veículo** passará a apontar para **null**, ou seja, deixará de referenciar qualquer objeto.
- O veículo não terá mais referência para o objeto Pessoa.
- Se tentarmos acessar algo como `veiculo.getPessoa().getNome()`, ocorrerá um erro (*NullPointerException*).



Relacionamento entre Classes

Agregação (relação “Tem um” - Fraca)

Relação “Todo-Parte” (fraco)



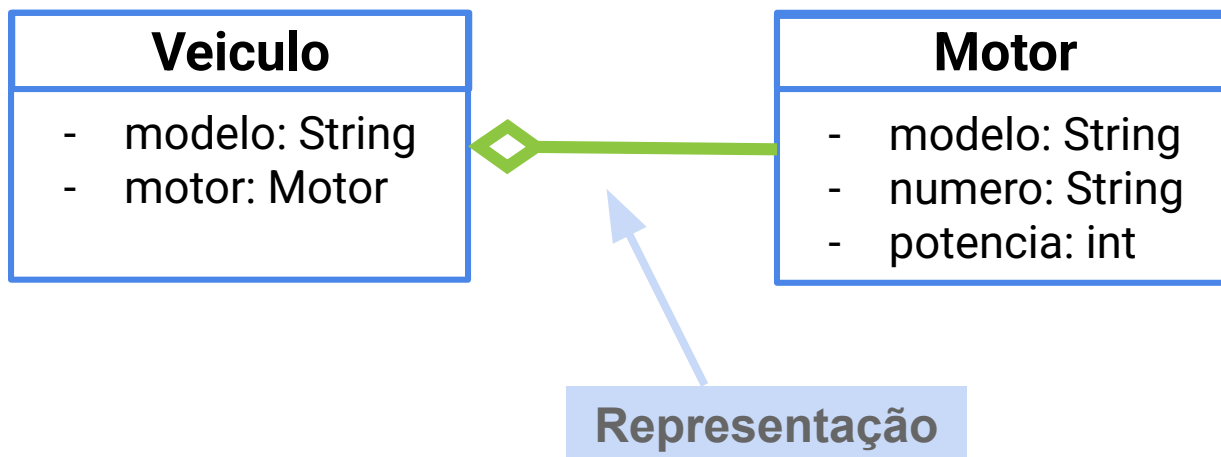


Relacionamento entre Classes

Agregação (relação “Tem um” - Fraca)

Na agregação, uma classe (o todo) possui referência para objetos de outra classe (as partes).

- Relação fraca: a “parte” pode existir sem o “todo”.



Se o veículo for destruído, o motor segue existindo e pode ser usado em outro veículo.



Relacionamento entre Classes

Agregação (relação “Tem um” - Fraca)

Todo

```
public class Veiculo {  
  
    private String modelo;  
    private Motor motor;  
  
    public Veiculo(String modelo, Motor motor) {  
        this.modelo = modelo;  
        this.motor = motor;  
    }  
}
```

Parte

```
public class Motor {  
    private String modelo;  
    private String numero;  
    private int potencia;  
}
```

O motor é passado pronto (agregação)

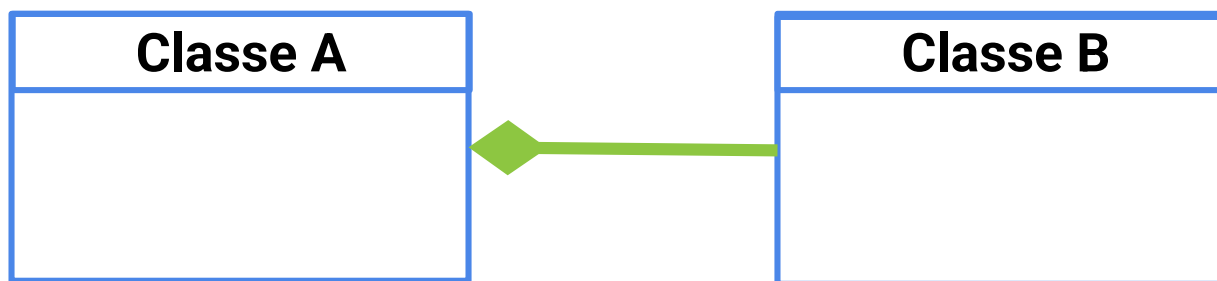
```
Motor m1 = new ("abc", "022929237", 70);  
Veiculo v1 = new Veiculo("Fusca", m1);
```



Relacionamento entre Classes

Composição (relação “Tem um” - Forte)

Relação “Todo-Parte” (forte)



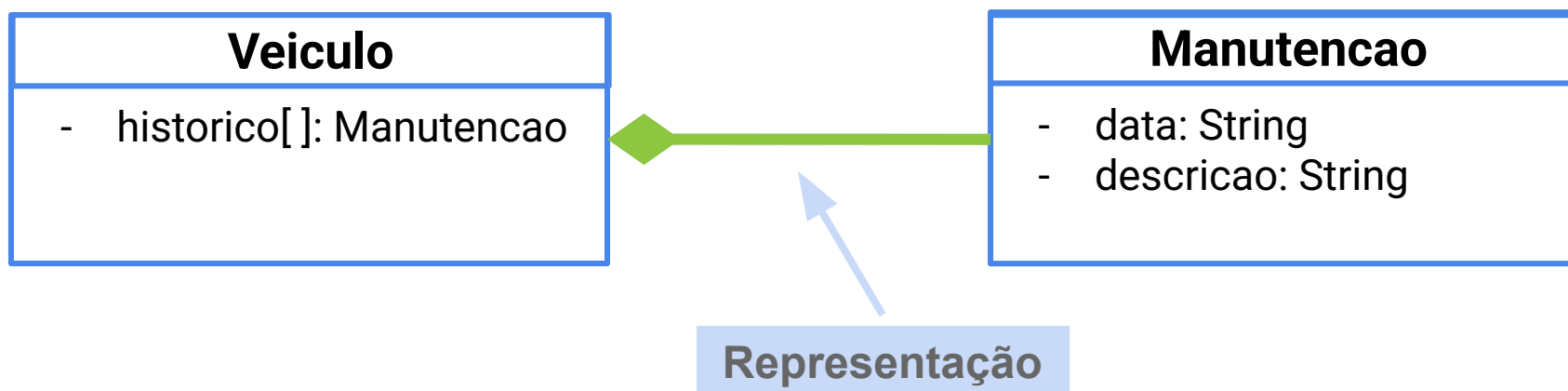


Relacionamento entre Classes

Composição (relação “Tem um” - Forte)

A composição é uma variação mais forte da agregação.

- As “partes” **dependem** do ciclo de vida do “todo”. A destruição do objeto contêiner (todo) implica na destruição automática de todos seus componentes (partes).





Relacionamento entre Classes

Composição (relação “Tem um” - Forte)

Todo

```
public class Veiculo {  
    private Manutencao[] historico;  
  
    public Veiculo() {  
        this.historico = new Manutencao[10];  
    }  
  
    public void registrarManutencao(String data, String descricao) {  
        historico[x] = new Manutencao(data, descricao);  
    }  
}
```

Parte

```
public class Manutencao {  
    private String data;  
    private String descricao;  
}
```

Estrutura que guarda a referência da(s) “partes”

A “parte” é instanciada e sua referência é atribuída ao “todo”.



Relacionamento entre Classes

Observação - Agregação VS Composição

Definição e Ciclo de Vida

A definição quanto ao tipo de relacionamento (agregação ou composição) depende das regras de negócio e contexto a que se destina o sistema. Exemplo:

- Se o sistema é destinado ao contexto de uma oficina mecânica, o **veículo** pode ser submetido a uma manutenção, ter seu **motor** removido e substituído por outro. No sistema, ao deletarmos **veículo**, o objeto **motor** continua existindo. Temos então uma **agregação** (Veículo, Motor).
- Se o sistema é destinado ao contexto de uma fábrica de automóveis, cada novo **veículo** fabricado (instância) obrigatoriamente deve possuir um **motor** em sua “certidão de nascimento”. No sistema, se o projeto do **veículo** for descartado, o registro daquele **motor** específico (vinculado àquela montagem) morre com ele. Temos assim uma **composição** (Veículo e Motor).



Relacionamento entre Classes

Herança ou Generalização (relação “É um”)





Relacionamento entre Classes

Herança ou Generalização (relação “É um”)

Permite que uma subclasse aproveite o código da superclasse.

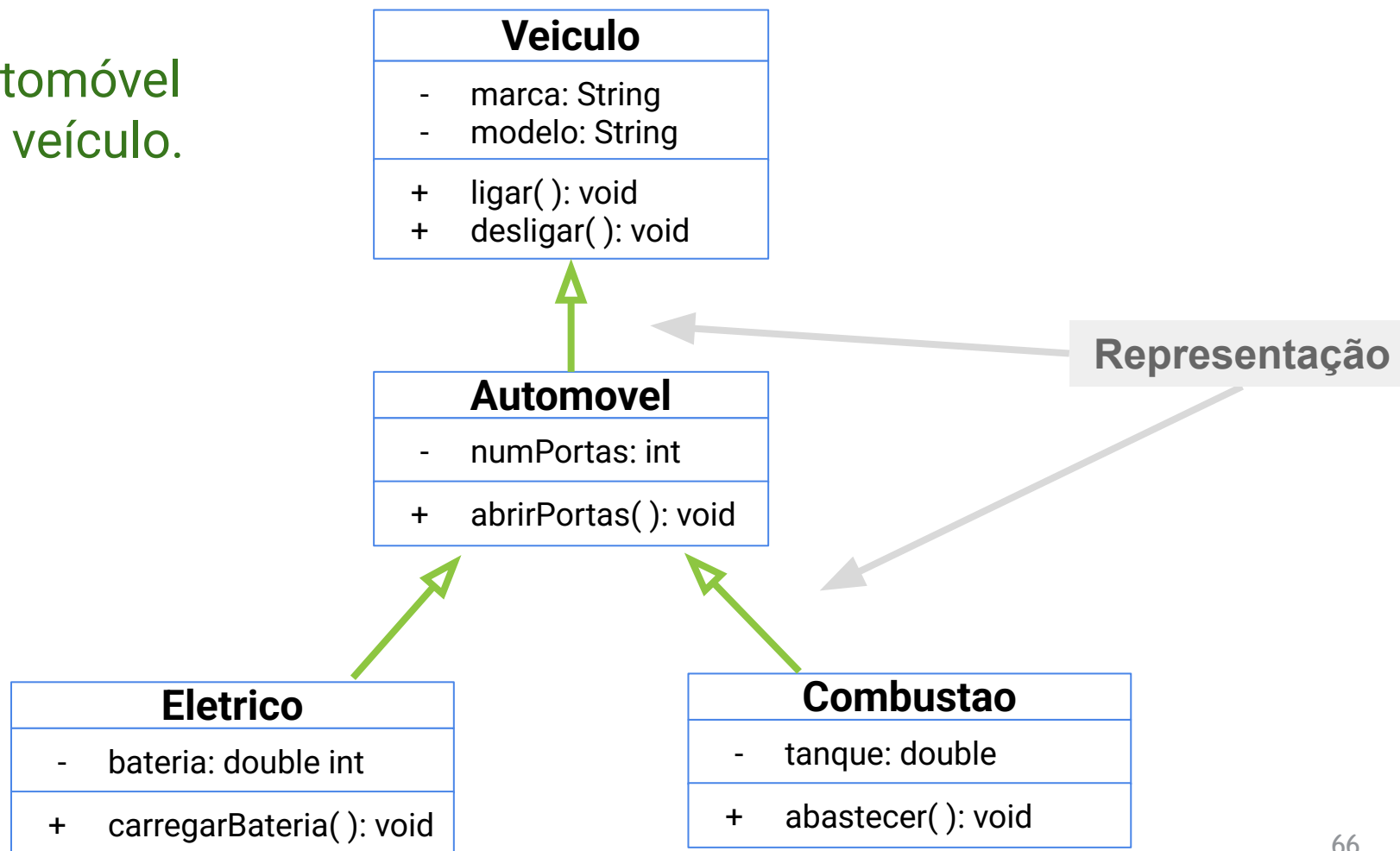
- Representa uma relação “é um(a)”
- Usa a palavra **extends** no java



Relacionamento entre Classes

Herança ou Generalização (relação “É um”)

- Elétrico é um Automóvel
- Automóvel é um veículo.





Relacionamento entre Classes

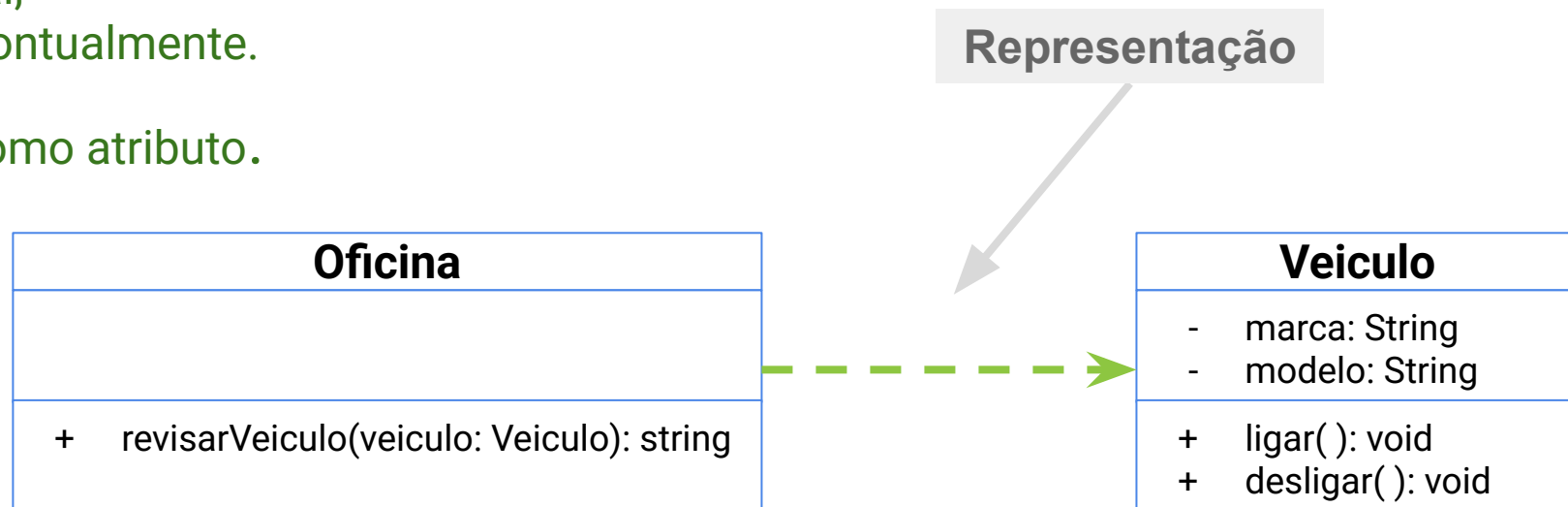
Dependência (relação “Usa um”)

É um tipo de relacionamento em que uma classe utiliza outra de forma **temporária**, sem manter referência permanente a ela.

Ocorre quando uma classe:

- recebe outra como parâmetro;
- a utiliza como variável local;
- ou invoca seus métodos pontualmente.

A classe não possui o objeto como atributo.





Relacionamento entre Classes

Dependência (relação “Usa um”)

```
public class Oficina {  
    // atributos omitidos  
  
    public void revisarVeiculo(Veiculo v) {  
        System.out.println("Iniciando revisão...");  
  
        v.ligar();  
        v.desligar();  
  
        System.out.println("Revisão concluída!");  
    }  
}
```

```
public class Veiculo {  
    private String marca;  
    private String modelo;  
  
    public Veiculo(String marca, String modelo) {  
        setMarca(marca);  
        setModelo(modelo);  
    }  
  
    public void ligar() {  
        // implementação suprimida  
    }  
  
    public void desligar() {  
        // implementação suprimida  
    }  
  
    //métodos get e set
```



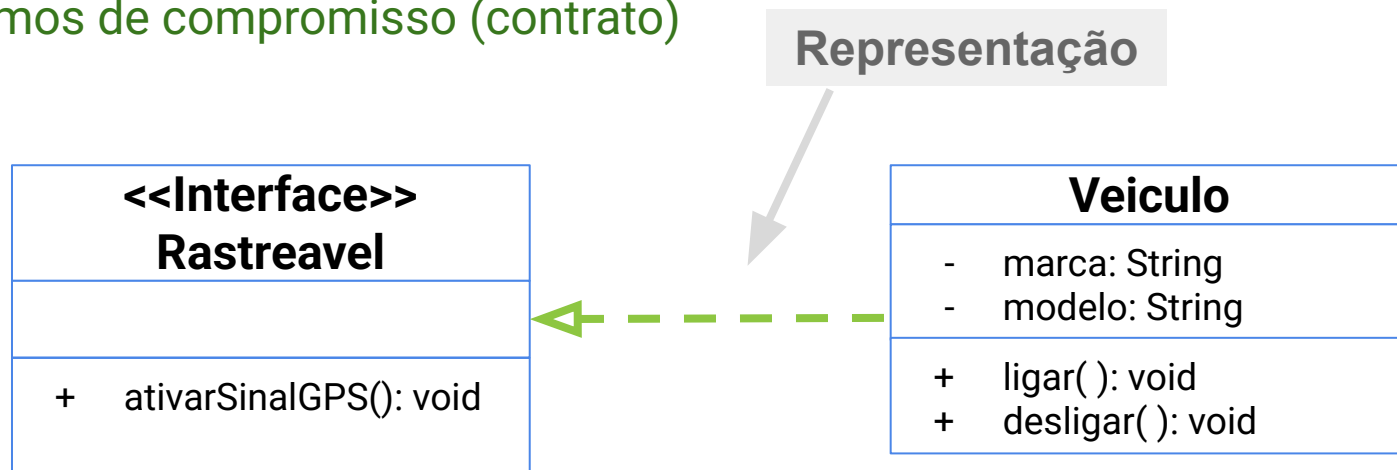
Relacionamento entre Classes

Realização ou Implementação

É um tipo de relacionamento de realização que funciona como um **contrato**, definindo um conjunto de ações e comportamentos que uma classe deve possuir, sendo a implementação de acordo com o contexto.

Ocorre quando uma classe:

- Assina um ou mais termos de compromisso (contrato)



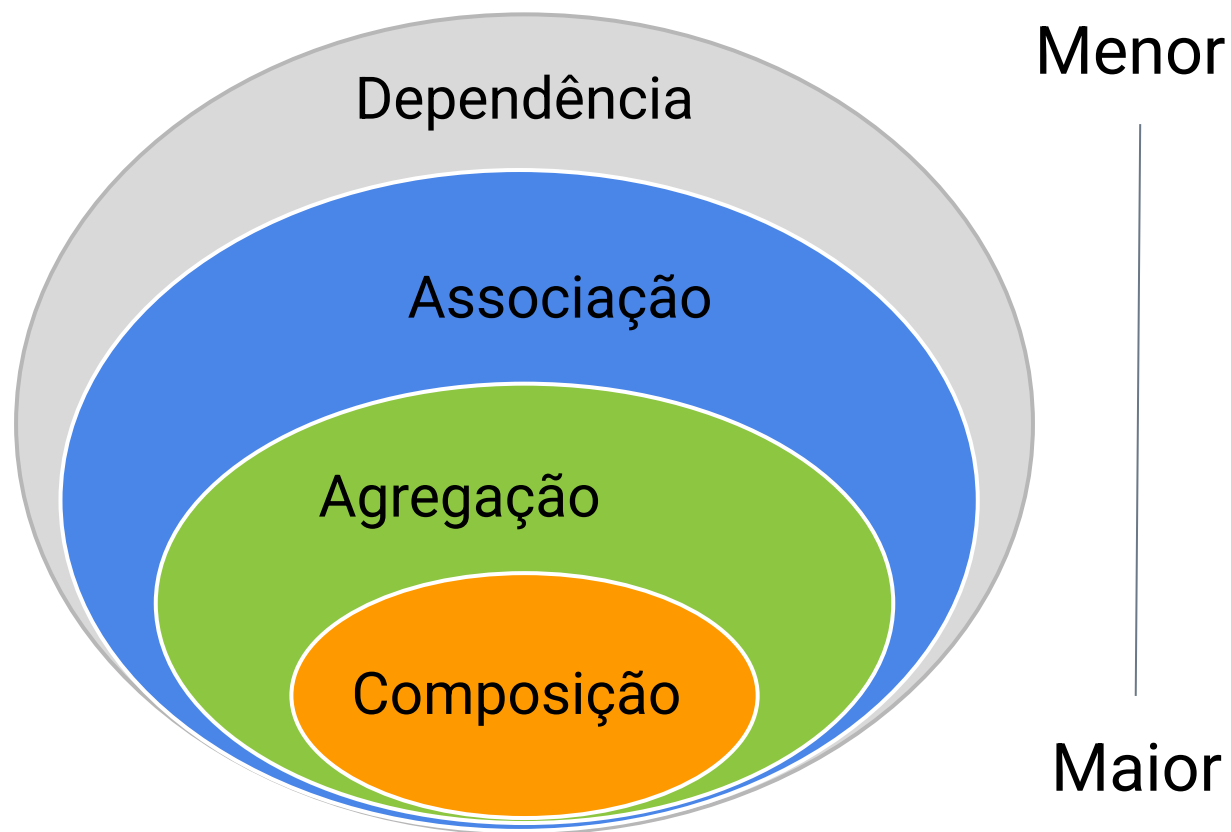


Relacionamento entre Classes

Nível de Acoplamento

A composição deve ser usada com cautela, pois ela cria o vínculo mais forte possível entre dois objetos.

Um dos princípios fundamentais do projeto orientado a objetos é buscar sempre o **menor nível de acoplamento possível**.





Relacionamento entre Classes

Multiplicidade

Define quantos objetos de uma classe podem se associar a um objeto de outra classe.

1..1	0..1	1..*	0..*
Um para Um	Zero para Um	Um para Muitos	Muitos
Relacionamento obrigatório e exclusivo entre dois objetos.	O relacionamento é opcional (pode ser nulo).	A associação exige pelo menos uma instância da parte.	Flexibilidade total: de nenhum a infinitos objetos.
<i>Ex: Veículo possui 1 Chassi</i>	<i>Ex: Carro e Seguro</i>	<i>Ex: Pedido e Itens</i>	<i>Ex: Pessoa e Telefones</i>



Exercícios Moodle



Chegou a sua vez, mão na massa!



POO

Classe Abstrata



Java

abstract class



POO

Classes Abstratas

O conceito de classe abstrata surge quando identificamos no nosso sistema uma entidade que é **genérica** demais para existir por si só no mundo real. Ou seja, aplicamos esse recurso quando não é desejável que se crie objetos diretos desse tipo.

Ela funciona estritamente como uma **superclasse conceitual**, servindo de base para a construção de novas classes que herdam seus atributos (estado) e comportamentos (métodos).



POO

Classes Abstratas

Por natureza é uma estrutura incompleta, não sendo possível invocar seu construtor para instanciá-la. Essa característica é determinada pela presença de métodos abstratos no escopo da superclasse.

Classe abstrata

```
public abstract class Veiculo {  
    private String marca;  
  
    public Veiculo(String marca) {  
        this.marca = marca;  
    }  
  
    public abstract void acelerar();  
}
```

método abstrato



```
public class Principal {  
    Run | Debug  
    public static void main(String[] args) {  
        Veiculo v = new Veiculo("marca");  
    }  
}
```

Instanciação direta
impossível



POO

Método Abstrato

Um **método abstrato** atua como uma declaração de assinatura obrigatória, definindo qual operação deve existir, mas sem especificar nenhum bloco de código para sua execução.

As subclasses que estendem essa estrutura denominam-se **classes concretas**, e são **obrigadas a implementar e codificar** os métodos abstratos do zero. Somente estas classes que possuem maturidade para serem instanciadas.

```
public abstract void acelerar();
```



POO

Método Abstrato

E se a instrução **abstract** não for usada na declaração do método?



POO

Método Não Abstrato

Primeiro, a linguagem passará a exigir que esse método tenha um corpo de implementação, ou seja, você será obrigado a abrir e fechar chaves { } na própria superclasse, mesmo que o bloco de código fique totalmente vazio ou contenha apenas um comportamento genérico. Ela deixa de ser uma mera "assinatura" e passa a ser uma função concreta e funcional.

Segundo, as subclasses **perdem a obrigatoriedade** de implementação. Como a superclasse já forneceu uma versão do método (mesmo que vazia), a classe filha simplesmente a herda e pode usá-lo exatamente como está ou modificá-lo (sobrescrever).



POO

Classe Abstrata

Classe abstrata

```
public abstract class Veiculo {  
    private String marca;  
  
    public Veiculo(String marca) {  
        this.marca = marca;  
    }  
  
    public abstract void acelerar();  
  
    public void frear(){  
        System.out.println("Veiculo em frenagem...");  
    }  
}
```

Método abstrato

Método comum



POO

Classe Concreta

```
public class Onibus extends Veiculo {  
    private int numAssentos;  
  
    public Onibus(String marca) {  
        super(marca);  
        this.numAssentos = 50;  
    }  
  
    @Override  
    public void acelerar() {  
        // implementação obrigatoria  
    }  
  
    @Override  
    public void frear(){  
        System.out.println("O ônibus está parando...");  
    }  
}
```

Classe concreta

Método **abstrato** deve
ser implementado

Método comum pode
ser modificado
(opcional)



POO

Classe Abstrata



Regra de Ouro do Compilador

Uma função abstrata **só pode ser declarada** dentro de uma classe que também seja abstrata. A presença de um único elemento incompleto obriga a classe inteira a assumir essa identidade, bloqueando preventivamente sua instanciação para garantir a integridade do sistema.



POO

Classes e Métodos Abstratos no Mundo Real

Onde vou utilizar classe abstrata?

- Na prática, utilizamos classes abstratas sempre que precisamos criar um molde obrigatório para coisas que são muito genéricas para existirem sozinhas no sistema.

Imagine o software de uma oficina mecânica. Nele, podemos ter uma superclasse abstrata chamada Veículo, que possui um método abstrato chamado realizarRevisao. No mundo real, um mecânico nunca conserta um 'veículo genérico'; ele atende um carro, uma motocicleta ou um caminhão. Portanto, não faz sentido permitir a criação de um objeto Veículo direto na memória. Em vez disso, o sistema obriga que as classes concretas (como Carro ou Moto) escrevam o código do método realizarRevisao do zero. Assim, o programa garante que todo transporte passe pela manutenção, mas respeita o fato de que as peças e os procedimentos mudam completamente dependendo de qual modelo real está sendo atendido.



POO

Tipo Abstrato - Objeto

Embora uma classe abstrata não possa ser instanciada, é possível declarar uma variável cujo tipo é uma classe abstrata, mas a instância aponta para uma classe concreta.

Exemplo:

```
public class Principal {  
    Run | Debug  
    public static void main(String[] args) {  
  
        Veiculo v;  
        v = new Onibus(marca: "abc");  
    }  
}
```

v é do tipo **Veiculo**, mas recebe uma instância de **Onibus**

```
public abstract class Veiculo {  
  
    public abstract void acelerar();  
  
    public void frear(){  
        System.out.println("Veiculo em frenagem...");  
    }  
  
    public abstract void realizarManutencao();  
}
```

```
public class Onibus extends Veiculo {  
    private int numAssentos;  
  
    @Override  
    public void acelerar() {  
        // implementação obrigatória  
    }  
  
    public void abrirJanelaEmergencia(){  
  
    }  
  
    @Override  
    public void realizarManutencao() {  
        //implementação obrigatória  
    }  
}
```



POO

Tipo Abstrato - Objeto

O que o sistema "enxerga"?

Ao declarar a variável como **Veiculo**, o Java só permite acessar os métodos que existem na superclasse abstrata. Métodos e variáveis exclusivas do **Onibus** ficam ocultos.

O que o sistema "executa"?

Se o método existir na superclasse e for chamado, o Java irá executar a versão específica da classe concreta no momento em que o programa rodar.



POO

Tipo Abstrato - Objeto

Cenário de uso:

Imagine a mesma oficina do exemplo anterior. Em vez de criar um método para receber cada especialização de Veiculo (registrarCarro, registrarCaminhao, etc.), criamos um único método que aceita objetos do tipo Veiculo - superclasse:

```
public class Oficina {  
    public void registrarVeiculo(Veiculo v){  
  
    }  
}
```

Aceita qualquer coisa que
estender de Veiculo



POO

Tipo Abstrato - Objeto

Se o veículo recebido for um Onibus e a oficina precisar realizar uma checagem exclusiva, usamos **if (v instanceof Onibus)** para validar a identidade do objeto e liberar acesso ao comportamento do modelo via **cast**.

```
public void registrarVeiculo(Veiculo v){  
    if(v instanceof Onibus){  
        Onibus onibus = (Onibus) v; //casting  
  
        onibus.abrirJanelaEmergencia(); //acesso a comportamento específico  
    }  
}
```



POO

Tipo Abstrato - Objeto

Cenário de uso 2:

Para organizar os transportes que aguardam atendimento na oficina, declaramos um único array tradicional do tipo genérico: **`Veiculo[ ] patio = new Veiculo[30]`**

Esse vetor armazena, de forma mista e na mesma estrutura, instâncias de Carro, Moto e Onibus. Ao percorrer o array, o sistema chama o método `realizarManutencao()` de cada posição, e o Java executa o comportamento polimórfico correto com base no objeto real que está ocupando aquela vaga na memória.



POO

Interface



Java

Interface

Definindo contratos de comportamentos



POO

Interface

Funciona como um **contrato** que define os **comportamentos** que uma classe deve obrigatoriamente fornecer, promovendo padronização entre objetos **sem relação direta** de herança.

Esse recurso é fundamental no Java, uma vez que a linguagem restringe a herança a apenas uma superclasse por vez, mas permite a implementação simultânea de **múltiplas interfaces**. Dessa forma, a arquitetura do sistema ganha flexibilidade, passando a se basear no que o objeto é capaz de fazer, independentemente de sua origem.





POO

Interface

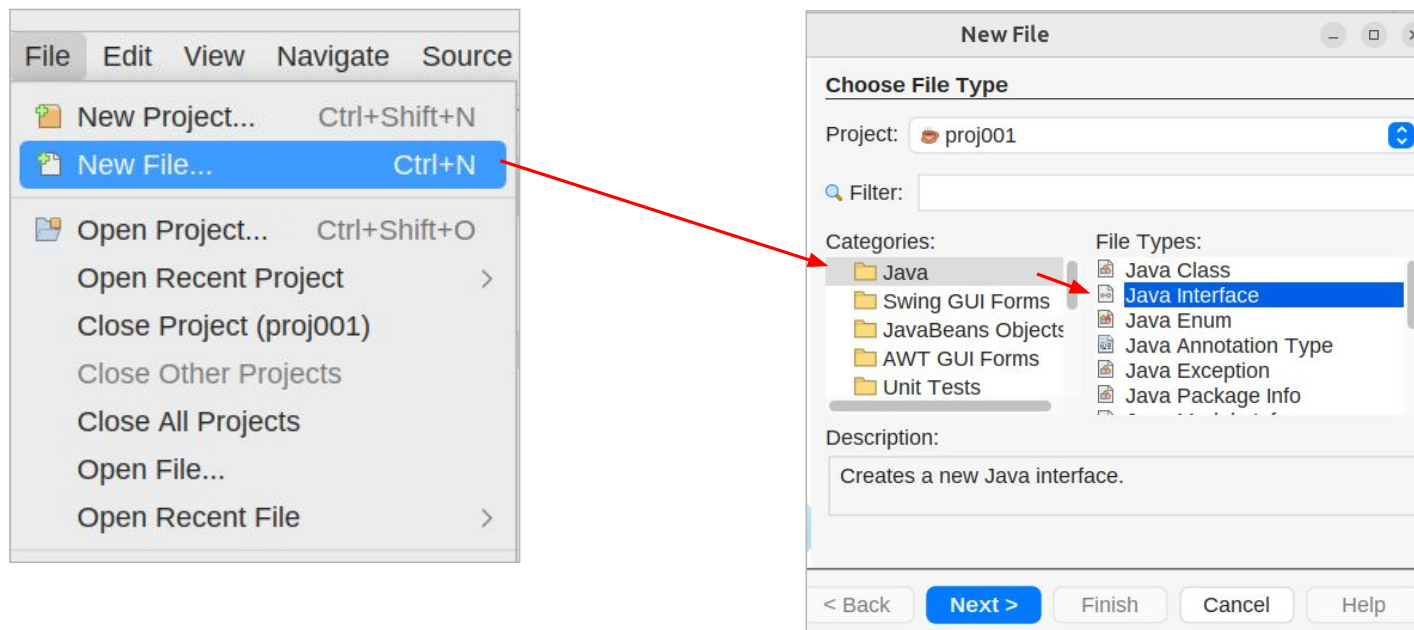
Até o Java 7, interfaces definiam apenas assinaturas de métodos e constantes, sem implementação.

A partir do Java 8, passaram a permitir métodos *default* e *static*, possibilitando a definição de comportamentos padrão reutilizáveis. Nesse caso, a implementação desses métodos na classe torna-se opcional (não é obrigatório sobrescrever).



POO

Interface no NetBeans IDE



Para definir o nome da interface, pense no que ela representa:

- Se ela define uma **capacidade**, use um adjetivo (Autenticavel, Renderizavel, Rastreavel).
- Se ela define um **papel ou serviço**, use um substantivo (GerenciadorDeContas, ProcessadorDePagamento).



POO

Definindo uma Interface

```
public interface Rastreavel {
```

Assinaturas

```
    public void ativarSinalGPS();  
    public void desativarSinalGPS();  
    public String obterCoordenadas();
```

Implementação
obrigatória

Função *Default*

```
    public default void emitirAlertaSOS() {  
        System.out.println("Iniciando protocolo de emergência...");  
        ativarSinalGPS();  
        System.out.println("Alerta: Pedido de socorro enviado para a central");  
    }
```

Implementação
opcional na
classe concreta

public não é obrigatório definir explicitamente nas assinaturas da Interfaces

SOS: Sinal Internacional de Socorro padronizado para indicar que uma pessoa, embarcação ou aeronave está em situação de grave perigo e necessita de assistência imediata



POO

Classe Concreta que Implementa a Interface

Implementação
Obrigatória

```
public class Carro extends Veiculo implements Rastreavel {  
  
    @Override  
    public void ativarSinalGPS() {  
        System.out.println("Antena GPS do teto ativada. Buscando satélites...");  
    }  
  
    @Override  
    public void desativarSinalGPS() {  
        System.out.println("Antena GPS desligada para poupar bateria do carro.");  
    }  
  
    @Override  
    public String obterCoordenadas() {  
        return "Retornando coordenadas atuais";  
    }  
    // outros métodos omitidos  
}
```

Também **extends** Veiculo

Assina o contrato



POO

Classe Concreta que Implementa a Interface

```
@Override  
public String obterCoordenadas() {  
    return "Retornando coordenadas atuais...";  
}
```

E as variáveis que contemplam a implementação da interface?

- A interface não tem memória, ou seja, não possui atributos de instância para armazenar o estado de um objeto. Apenas define as funções (contrato).
- O trabalho de guardar os dados reais (ex.: latitude e longitude), bem como manipular esses valores fica inteiramente a cargo da classe concreta.

```
public class Carro extends Veiculo implements Rastreavel {  
    { private double latitude;  
      private double longitude;
```

```
@Override  
public String obterCoordenadas() {  
    return "Latitude: " + this.latitude + " Longitude: " + this.longitude;  
}
```



POO

Analogia

*Podemos imaginar uma tomada da parede como uma **Interface**. Ela define um contrato rígido (o formato dos três pinos e a voltagem). A tomada tem um **baixo acoplamento** com os aparelhos da sua casa. Ela não quer saber se você vai plugar uma televisão, uma geladeira ou um carregador de celular. Desde que o aparelho 'assine o contrato' (tenha o plugue no formato correto), a energia vai fluir. A interface Rastreavel faz a mesma coisa: ela não se importa com qual veículo está lidando, desde que ele tenha o método de ligar o GPS.*



POO

Constantes

São “variáveis” cujo valor não pode ser alterado depois de ser definido pela primeira vez. Ou seja, é um valor fixo durante toda a execução do programa.

Para criar uma constante em uma classe Java, é necessário duas palavras-chave: ***static*** e ***final***.

Em ***interfaces***, uma variável (constante) será sempre ***public***, ***static*** e ***final***, mesmo que isso não esteja declarado explicitamente.



POO

Interface com Variáveis (Constantes)

static e final
implicitamente
presentes

Padrão declaração:
UPPER_SNAKE_CASE

```
public interface Rastreavel {  
    { public int INTERVALO_ATUALIZACAO_SEGUNDOS = 10;  
      public String PROTOCOLO_COMUNICACAO = "NMEA-0183";  
  
    { public default void emitirAlertaSOS() {  
      System.out.println("Iniciando protocolo de emergência...");  
      ativarSinalGPS();  
      System.out.println("Alerta: Pedido de socorro enviado via " + PROTOCOLO_COMUNICACAO);  
    }  
}
```

Método *Default*



POO

Na Classe Concreta

```
public class Carro extends Veiculo implements Rastreavel {  
  
    @Override  
    public void ativarSinalGPS() {  
        System.out.println("Antena GPS ativa. Buscando satélites...");  
        System.out.println("Atualizando localização a cada " + INTERVALO_ATUALIZACAO_SEGUNDOS + " segundos");  
    }  
}
```

Acesso a constante



POO

Casos de Uso

A mesma interface (contrato) “*Rastreavel*” do presente exemplo, pode ser implementada a outros cenários que também requeiram rastreabilidade:

- Celular
- SmartWatch
- Computador
- VANTs (Veículos Aéreos Não Tripulados)

Classes



POO

Static



Java

Modificador Static



POO

Static

O *static* é um **modificador de não-acesso** em Java. Ele indica que um membro pertence à própria classe (o molde), e não a uma instância específica (o objeto) gerada a partir dela.

Possui escopo global, ou seja, existe apenas **uma única cópia** na memória, independentemente de quantos objetos foram instanciados a partir dessa classe.



POO

Static

O Modificador *static* aplica-se a:

- ❖ Atributos
- ❖ Métodos
- ❖ Classes Aninhadas (Classes criadas dentro de outras classes)
- ❖ Blocos de inicialização



POO

Método Estático

Pode ser invocado diretamente pelo nome da classe, sem usar o ***new***

Classe Math

```
double potencia = Math.pow(2, 3);  
double maximo = Math.max(10, 12);  
double raiz = Math.sqrt(16);  
int numAleatorio = (int) (Math.random() * 51);
```

[Saiba mais](#)

Chamada direta via Classe

Quando utilizar?

Quando o método realiza uma tarefa que não depende do estado (valores) de um objeto (instância) específico.



POO

Método Estático - Caso de Uso

Exemplo, para validar o formato de uma placa, não faz sentido gastar memória instanciando um objeto Carro inteiro apenas para checar um texto. Como essa regra de formatação é universal, usamos o *static* para que o método pertença à própria Classe. Assim, realizamos a validação direta, sem a necessidade de criar o objeto em memória.

```
public abstract class Veiculo {  
    public static boolean validarFormatoPlaca(String placaDigitada) {  
        if (placaDigitada != null && (placaDigitada.length() == 7 || placaDigitada.length() == 8)) {  
            return true;  
        }  
        return false;  
    }  
}
```




POO

Atributo Estático

```
public abstract class Veiculo {  
    private static int totalVeiculosFabricados = 0;  
  
    private String marca;  
    private String modelo;  
    private int ano;  
  
    public Veiculo(String marca, String modelo, int ano) {  
        Veiculo.totalVeiculosFabricados++;  
  
        this.marca = marca;  
        this.modelo = modelo;  
        this.ano = ano;  
    }  
  
    public static int getTotalVeiculosFabricados() {  
        return totalVeiculosFabricados;  
    }  
}
```

Não utiliza "this"

pertence a classe não a instância

A cada novo veiculo fabricado (new), a variável estática incrementa contagem



POO

Atributo Estático - Caso de Uso

Imagine que uma classe **Fabrica** produziu 1000 objetos de uma classe **Carro**. Se o atributo que estabelece a tolerância para primeira revisão do carro for de instância (nasce com objeto), então teremos mil variáveis em memória com mesmo conteúdo. Se tornarmos esse atributo estático, apenas um espaço de memória será reservado e fica visível para todos objetos instanciados.

```
public class Carro extends Veiculo implements Rastreavel {  
  
    private int numPortas;  
    private int id;  
    private int quilometragemAtual;  
    private static int toleranciaRevisaoKm = 1000;  
}
```

Atributo de classe



POO

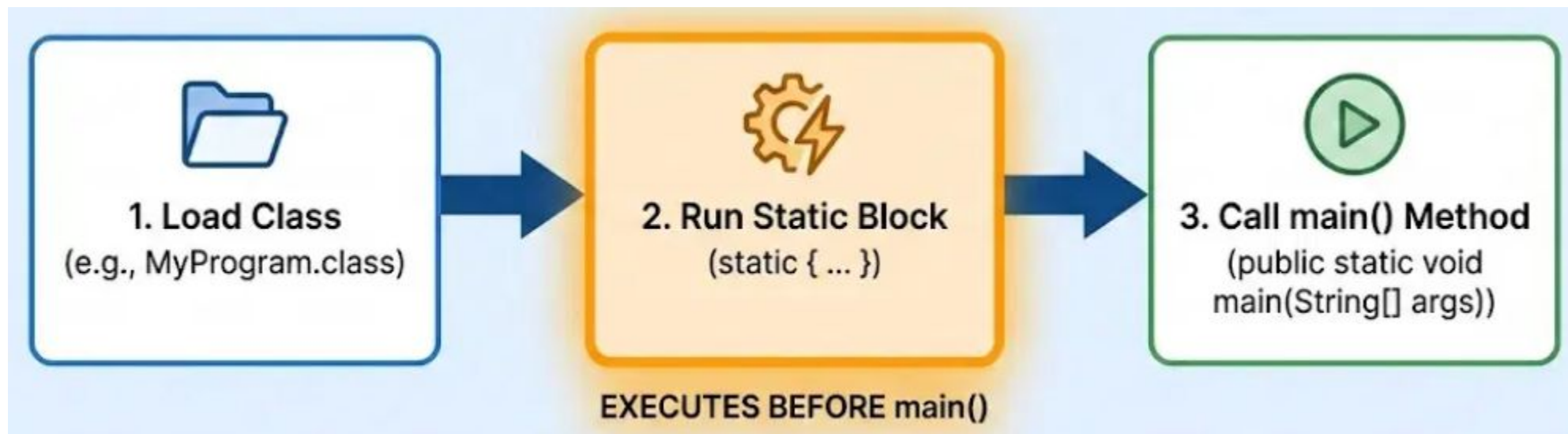
Estático Vs. Instância

Característica	Membro Estático	Membro de Instância
Pertence a	À Classe (ao Molde)	Ao Objeto (Instância)
Chamada	<code>NomeDaClasse.membro</code>	<code>objeto.membro</code>
Memória	Uma única cópia global	Uma cópia por objeto
Acesso	Apenas outros membros estáticos	Qualquer membro



POO

Na JVM



Saiba mais



POO

Static, onde aplicar?

- Constantes universais da aplicação (combinado com *final*)

```
private static final double pi = 3.14;  
private static final String URL_DOCUMENTO = "/home/diretorio/doc";
```

- Métodos utilitários e validadores;
- Controle de recursos e contadores globais



POO



Dúvidas?



luizwasielewski@ifsul.edu.br